

LAGMX: Automated preparation and simulation of single- and multi-ligand protein complexes in GROMACS

La Ode Aman¹, Arfan², Aiyi Asnawi³, Purnawan Pontana Putra⁴, Netty Ino Ischak⁵, Dizky Ramadani Putri Papeo¹, Hamsidar Hasan¹, and A. Mu'thi Andy Suryadi¹

¹ Department of Pharmacy, Faculty of Sports and Health, Universitas Negeri Gorontalo, Jl. Jenderal Sudirman No. 6, Kota Gorontalo, Gorontalo 96128, Indonesia ² Faculty of Pharmacy, Universitas Halu Oleo, Kendari, Southeast Sulawesi, Indonesia ³ Faculty of Pharmacy, Universitas Bhakti Kencana, Bandung, West Java, Indonesia ⁴ Department of Pharmaceutical Chemistry, Faculty of Pharmacy, Universitas Andalas, Padang 25163, Indonesia ⁵ Department of Chemistry, Faculty of Mathematics and Natural Sciences, Universitas Negeri Gorontalo, Gorontalo, Indonesia ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 02 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

LAGMX is a single Python script that automates the preparation, equilibration, and production stages of a molecular dynamics (MD) simulation of a protein-ligand complex in GROMACS (Abraham et al., 2015). Preparing one such system by hand normally requires nine manual stages – receptor structure curation, ligand parameterization, topology merging, solvation, ion addition, energy minimization, index-group selection, and two stages of equilibration – several of which require interactive input. LAGMX runs all of this, plus a configurable production run, from one invocation, for every complex directory it is pointed at.

The tool is built specifically to handle **multiple ligands per complex correctly**, whether those ligands are identical copies, distinct compounds, or charged species. Correctness problems specific to this setting – moleculetype name collisions between identical ligand copies, position-restraint includes attached at the wrong scope in the merged topology, duplicated [atomtypes] entries, and per-ligand net charge that is silently assumed to be neutral – are handled automatically rather than left to the user to discover through failed or silently incorrect runs. Ligand parameterization uses GAFF2 with a configurable charge method (AM1-BCC via antechamber (Wang et al., 2006) for production accuracy, or Gasteiger charges for fast iteration), and force-field parameters follow the GAFF methodology (Jakalian et al., 2002; Wang et al., 2004).

Production length is a single configuration value; an interrupted or deliberately extended run resumes from the GROMACS checkpoint rather than restarting the whole pipeline, and the tool detects and skips whichever pipeline stages (preparation, energy minimization, NVT, NPT) a given complex has already completed.

Statement of need

Protein-ligand MD is routine in structure-based drug discovery, but the setup cost per system is high and scales badly with the number of ligands. A researcher screening a series of candidate compounds, or studying a receptor with several co-bound cofactors, repeats nine largely manual stages for every complex, several of which prompt for interactive input. The multi-ligand failure modes are quiet rather than loud: GROMACS accepts a topology in which two identical ligand copies share a moleculetype name, and antechamber accepts a charged ligand as neutral. Both produce a simulation that runs to completion and is wrong.

LAGMX is aimed at computational chemists and pharmacy researchers who need many such systems prepared reproducibly, and who would rather not maintain a private pipeline of shell glue to get them. It reduces the setup of an arbitrary number of complexes to editing one configuration file and issuing one command.

State of the field

Several existing tools address parts of this workflow, but none combine automated, config-driven, end-to-end execution with first-class support for an arbitrary number of ligands per complex:

- **CHARMM-GUI** (Jo et al., 2008) is a widely used web service that generates ready-to-run input files for a protein-ligand system, including multi-ligand systems. It does not itself run the MD pipeline: preparation happens on CHARMM-GUI's servers, and equilibration/production still have to be launched and managed by the user on their own machine.
- **acpype** (Sousa da Silva & Vranken, 2012) automates ligand topology generation (a wrapper around antechamber), but does not merge topologies, solvate, equilibrate, or run production – it addresses one stage of the pipeline, not the pipeline itself.
- **BioExcel Building Blocks (BioBB)** (Andrio et al., 2019) provides a large library of composable Python wrappers around individual GROMACS/AmberTools commands (gmx solvate, gmx genion, gmx grompp, and others). It gives users the building blocks to construct a pipeline, rather than a pipeline that runs by default; assembling and validating that pipeline – including the multi-ligand-specific correctness issues above – is left to the user.

LAGMX targets the gap between these: a single, config-driven script that a user with a set of receptor and ligand files can point at a directory and get a fully prepared, equilibrated, and simulated system out the other end, with the multi-ligand-specific failure modes handled by the tool rather than discovered by the user.

Software design

LAGMX is deliberately a single ~900-line script rather than a package, depending only on GROMACS, AmberTools, ParmEd, and the Python standard library. The target user typically works on a shared HPC node or a lab workstation where installing a package tree is friction rather than convenience, and a file that can be copied next to the data and run fits that setting better. The cost is that the tool is harder to unit-test and to reuse programmatically; this is a prototype trade-off we expect to revisit.

Configuration lives in one plain-text gmx_config.txt rather than in command-line flags. Every parameter that affects the physics – force field, water model, box type and margin, charge method, atom type, production length, mdrun resource options – is recorded in a single file that sits beside the results, so the settings that produced a trajectory remain recoverable months later without reconstructing a shell history. Flags would have made the common case shorter to type and the reproducible case harder.

Stage resumption is derived from output files rather than from a state database. detect_stage() decides how far a complex has progressed by testing for the final artefact of each stage (em.gro, nvt.gro, npt.gro, md.cpt), so a run interrupted midway through equilibration repeats only that stage and leaves the expensive topology construction and minimization intact. This keeps the tool stateless and robust to being killed, at the price of assuming the directory is not edited by hand between runs.

Two correctness decisions are worth naming. build_ligand_ids() assigns every ligand file a unique moleculetype name up front, appending a numeric suffix on collision, so identical ligand copies cannot silently share a [moleculetype] block. detect_net_charge() reads the formal-charge column of a PDB, or sums the partial charges of a MOL2 file, and passes the

result to antechamber -nc instead of letting it default to neutral. Both run on the input files before any GROMACS command executes, so a mis-specified system fails during preparation rather than yielding a trajectory that is wrong but plausible.

LAGMX did not begin as a single tool. Between July and August 2023 the first author published six single-purpose shell scripts, each automating one part of this workflow: autogmx_protein-ligand and autogmx_protein-2ligands for preparation and simulation, gmxana1_lig2 and gmxana2_lig2 for trajectory analysis and binding-energy calculation, ladina_liglig for ligand-ligand complexes, and extend_mdun for extending production runs. Each worked, but each handled one case, they shared no code, and the multi-ligand topology problem was solved separately and inconsistently in each. LAGMX consolidates that work into one implementation in which the topology logic exists exactly once. The earlier repositories remain publicly available as a record of that development, and are superseded by this software.

Validation

LAGMX ships with a version-controlled test matrix (run_matrix/) of seven scenarios exercising the parts of the pipeline most likely to break: a single receptor chain with one ligand; a receptor split across three files/chains with three distinct ligands; a single receptor file with three chains and three *identical* ligand copies (moleculetype deduplication); a ligand supplied as MOL2 instead of PDB; a charged ligand (ADP3-) alongside a neutral one; a deliberately malformed ligand, which is expected to fail cleanly rather than produce a silently wrong topology; and a five-ligand system mixing identical, distinct, and charged species.

All six valid scenarios have been run end to end – preparation through 5 ns of production – on a single consumer GPU (RTX 3070), with zero atom-name mismatches and zero LINCS constraint warnings across all of them. Energy minimization converged in every case; NPT temperature and density are reported per scenario in the project README.

Research impact statement

LAGMX has not yet been used outside the authors' group, and we make no claim of realized external impact. Its case rests on near-term significance and on being ready for others to pick up.

The validation matrix described above is the primary evidence. All seven scenarios, including their receptor and ligand inputs and their .mdp parameter files, are committed to the repository, so any reader can reproduce the reported runs from a clean checkout rather than take the numbers on trust. The deliberately malformed scenario is part of that matrix: it documents the boundary at which the tool refuses to proceed, so users can distinguish a genuine input problem from a tool failure.

For community readiness, the software is MIT-licensed, its test suite runs in continuous integration across Python 3.10-3.12, and it ships contribution guidelines. The workflow it automates – multi-ligand protein-ligand MD in GROMACS – is common enough in structure-based drug discovery that we expect the audience for a working end-to-end pipeline to be broader than our own group. Realized impact remains to be demonstrated, and we regard growing it as the main task ahead of this software rather than a settled result.

Limitations

LAGMX is at an early stage (prototype, not yet used outside this project). Five nanoseconds of production demonstrates that the pipeline executes correctly, not that any given system's dynamics have converged; grompp is still run with -maxwarn during equilibration and production; index-group selection assumes ligands are the only non-protein, non-solvent species present; and MOL2 ligand input is not guaranteed to be chemically complete – one test ligand required a missing hydrogen to be restored by a separate repair script before antechamber would accept

139 it. These limitations are stated explicitly in the project README and are the subject of
140 ongoing work.

141 AI usage disclosure

142 Generative AI tools were used in the preparation of both the software and this manuscript,
143 and are disclosed here in full.

144 **Software.** AI coding assistants were used for parts of LAGMX.py, including debugging, drafting
145 individual functions, and clarifying GROMACS and AmberTools command usage. The pipeline
146 architecture and the majority of the implementation were written by the authors. All AI-assisted
147 code was exercised by the authors against the validation matrix described above; the results
148 reported here were produced by running that code, not by inspecting it.

149 **Manuscript.** AI assistance was used for parts of the prose throughout the manuscript and
150 the project README, for drafting and for editing. In the revision that produced this version,
151 an AI coding assistant (Claude, Anthropic) drafted the State of the field, Software design,
152 Research impact statement, and AI usage disclosure sections, rewrote the Statement of need,
153 and authored the continuous integration workflow that compiles the manuscript.

154 **Verification.** All AI-assisted output was reviewed by the authors before inclusion. The design
155 decisions reported in Software design were checked against the source code; the validation
156 results were checked against the simulation outputs; and each reference was checked against
157 its DOI. The authors take full responsibility for the content of this paper and of the software.

158 Acknowledgements

159 This research received no external funding; it was self-funded by the authors.

160 References

- 161 Abraham, M. J., Murtola, T., Schulz, R., Páll, S., Smith, J. C., Hess, B., & Lindahl, E. (2015).
162 GROMACS: High performance molecular simulations through multi-level parallelism from
163 laptops to supercomputers. *SoftwareX*, 1-2, 19–25. [https://doi.org/10.1016/j.softx.2015.](https://doi.org/10.1016/j.softx.2015.06.001)
164 [06.001](https://doi.org/10.1016/j.softx.2015.06.001)
- 165 Andrio, P., Hospital, A., Conejero, J., Jordá, L., Del Pino, M., Codo, L., Soiland-Reyes, S.,
166 Goble, C., Lezzi, D., Badia, R. M., Orozco, M., & Gelpí, J. Ll. (2019). BioExcel building
167 blocks, a software library for interoperable biomolecular simulation workflows. *Scientific*
168 *Data*, 6, 169. <https://doi.org/10.1038/s41597-019-0177-4>
- 169 Jakalian, A., Jack, D. B., & Bayly, C. I. (2002). Fast, efficient generation of high-quality atomic
170 charges. AM1-BCC model: II. Parameterization and validation. *Journal of Computational*
171 *Chemistry*, 23(16), 1623–1641. <https://doi.org/10.1002/jcc.10128>
- 172 Jo, S., Kim, T., Iyer, V. G., & Im, W. (2008). CHARMM-GUI: A web-based graphical
173 user interface for CHARMM. *Journal of Computational Chemistry*, 29(11), 1859–1865.
174 <https://doi.org/10.1002/jcc.20945>
- 175 Sousa da Silva, A. W., & Vranken, W. F. (2012). ACPYPE - AnteChamber PYthon parser
176 interfacE. *BMC Research Notes*, 5, 367. <https://doi.org/10.1186/1756-0500-5-367>
- 177 Wang, J., Wang, W., Kollman, P. A., & Case, D. A. (2006). Automatic atom type and bond
178 type perception in molecular mechanical calculations. *Journal of Molecular Graphics and*
179 *Modelling*, 25(2), 247–260. <https://doi.org/10.1016/j.jmgm.2005.12.005>
- 180 Wang, J., Wolf, R. M., Caldwell, J. W., Kollman, P. A., & Case, D. A. (2004). Development
181 and testing of a general amber force field. *Journal of Computational Chemistry*, 25(9),
182 1157–1174. <https://doi.org/10.1002/jcc.20035>